

Architekturen und Anwendungen von KI-Systemen

DevFlowAI

Name: Blend Salihu

Matrikelnummer: 348128

Dozent: Dr. Sebastian Hobert

Inhaltsverzeichnis

1. Einleitung	1
2. Motivation und Use Case	1
2.1 Motivation	1
2.2 Use Case	2
3. Systemübersicht	2
4. Systemarchitektur	4
4.1 Frontend	4
4.2 Backend.....	4
4.3 KI-Modul	5
5. KI-Ansatz	5
5.1 Verarbeitung strukturierter Daten	6
5.2 Analyse unstrukturierter Daten (NLP)	6
5.3 Scoring- und Priorisierungsmodell	6
5.4 Erklärbarkeit der KI-Entscheidungen.....	7
5.5 Einordnung des KI-Ansatzes	7
6. Implementierung	7
6.1 Technologiewahl.....	8
6.2 Backend-Implementierung	8
6.3 KI-Modul	8
6.4 Frontend-Implementierung	9
6.5 Integration und Ablauf.....	9
6.6 Besonderheiten der Implementierung	9
7. Evaluation	9
7.1 Funktionale Evaluation	9
7.2 Bewertung des KI-Ansatzes	10
7.3 Erklärbarkeit und Transparenz	10
7.4 Nutzerperspektive.....	10
7.5 Gesamteinschätzung	11
8. Limitationen	11
9. Ausblick	12
10. Fazit	13
11. Quellen	13
12. Ergänzende Informationen	13
12.1 Gitlab Link von DevFlowAI:	14

1. Einleitung

In der modernen Softwareentwicklung müssen Entwickler häufig eine Vielzahl von Aufgaben parallel bearbeiten. Diese Aufgaben unterscheiden sich hinsichtlich Dringlichkeit, Wichtigkeit, Risiko und Aufwand. Die Priorisierung erfolgt dabei oftmals manuell und basiert auf subjektiven Einschätzungen oder individueller Erfahrung, was zu inkonsistenten oder suboptimalen Entscheidungen führen kann [4].

Ziel dieses Projekts ist die Konzeption und prototypische Implementierung von DevFlowAI, einem KI-gestützten System zur Aufgaben Priorisierung für Softwareentwickler. Das System analysiert Aufgabenmetadaten sowie natürlichsprachliche Beschreibungen und erzeugt daraus eine transparente und nachvollziehbare Prioritätenliste.

2. Motivation und Use Case

2.1 Motivation

In der heutigen Softwareentwicklung sind Entwickler zunehmend mit einer Vielzahl paralleler Aufgaben konfrontiert. Neben der eigentlichen Implementierung neuer Funktionalitäten fallen regelmäßig Tätigkeiten wie das Beheben von Fehlern, Refactoring bestehender Komponenten, das Schreiben von Tests sowie die Pflege von Dokumentation an. Diese Aufgaben unterscheiden sich erheblich hinsichtlich ihrer Dringlichkeit, ihres technischen Risikos und ihres erforderlichen Aufwands.

Die Priorisierung dieser Aufgaben erfolgt in der Praxis häufig manuell. Entwickler oder Teams entscheiden auf Basis persönlicher Erfahrung, subjektiver Einschätzungen oder kurzfristiger Anforderungen, welche Aufgaben zuerst bearbeitet werden. Dieses Vorgehen ist jedoch fehleranfällig. Kritische Aufgaben mit hohem Risiko können übersehen werden, während weniger relevante Tätigkeiten unverhältnismäßig viel Aufmerksamkeit erhalten. Zudem ist die Entscheidungsgrundlage für Außenstehende meist nicht transparent.

Ein weiterer Aspekt ist die steigende kognitive Belastung. Entwickler müssen gleichzeitig Deadlines im Blick behalten, den geschätzten Aufwand berücksichtigen und technische Risiken bewerten. Insbesondere in agilen Entwicklungsprozessen, in denen sich Anforderungen häufig ändern, wird diese Entscheidungsfindung zunehmend komplex.

Vor diesem Hintergrund bietet der Einsatz von Künstlicher Intelligenz großes Potenzial. KI-basierte Entscheidungsunterstützungssysteme können relevante Faktoren systematisch erfassen, gewichten und konsistent auswerten [4]. Sie ermöglichen eine objektivere und reproduzierbare Priorisierung und entlasten Entwickler bei der Entscheidungsfindung. Gleichzeitig können KI-Systeme ihre Entscheidungen nachvollziehbar begründen, was die Akzeptanz und das Vertrauen

in die Ergebnisse erhöht.

Das Projekt DevFlowAI greift diese Problematik auf und untersucht, wie ein KI-gestützter Ansatz zur Aufgabepriorisierung in der Softwareentwicklung prototypisch umgesetzt werden kann.

2.2 Use Case

Der zentrale Use Case von DevFlowAI ist die Unterstützung von Softwareentwicklern bei der Priorisierung ihrer täglichen Entwicklungsaufgaben. Das System ist darauf ausgelegt, typische Aufgaben aus dem Softwareentwicklungsalltag zu verarbeiten und diese anhand definierter Kriterien zu bewerten.

Die Zielgruppe des Systems sind einzelne Entwickler oder kleine Entwicklungsteams, die ihre Aufgaben strukturiert erfassen und priorisieren möchten. Dabei kann es sich beispielsweise um folgende Aufgabentypen handeln:

- Behebung von Softwarefehlern (Bugfixes)
- Refactoring bestehender Codebasis
- Implementierung neuer Funktionen
- Pflege von Dokumentation oder Tests

Für jede Aufgabe werden sowohl strukturierte Metadaten als auch unstrukturierte Textinformationen erfasst. Die Beschreibung der Aufgabe wird als Freitext eingegeben und kann technische Hinweise wie „Bug“, „Crash“ oder „Security“ enthalten. Zusätzlich werden numerische Werte für Wichtigkeit und Aufwand sowie eine Deadline angegeben.

Auf Basis dieser Informationen analysiert DevFlowAI die Aufgabe und berechnet einen Prioritätswert. Das Ergebnis ist eine sortierte Aufgabenliste, in der Aufgaben mit hoher Dringlichkeit und hohem Risiko entsprechend weiter oben erscheinen. Zusätzlich liefert das System zu jeder Aufgabe eine kurze Begründung, die erklärt, welche Faktoren maßgeblich zur Priorisierung beigetragen haben.

Der Use Case ist bewusst praxisnah gewählt, da er typische Entscheidungsprozesse in der Softwareentwicklung widerspiegelt. Gleichzeitig eignet er sich gut, um den Einsatz von KI-Techniken wie Textanalyse, heuristischer Bewertung und erklärbarer Entscheidungsfindung zu demonstrieren.

3. Systemübersicht

DevFlowAI ist als prototypisches, web-basiertes KI-System konzipiert, das Entwickler bei der Priorisierung von Softwareentwicklungsaufgaben unterstützt. Ziel des Systems ist es nicht, eine produktionsreife Anwendung bereitzustellen, sondern die grundlegenden Konzepte einer KI-gestützten Entscheidungsunterstützung in einer klaren und nachvollziehbaren Architektur zu

demonstrieren.

Das System folgt einem modularen Aufbau und gliedert sich in drei zentrale Komponenten: Frontend, Backend und KI-Modul. Diese Trennung ermöglicht eine klare Verantwortungsverteilung und erleichtert sowohl das Verständnis als auch die Erweiterbarkeit des Prototyps.

Das Frontend dient als Benutzerschnittstelle und stellt eine einfache Weboberfläche zur Verfügung, über die Nutzer Aufgaben erfassen können. Über ein Formular werden alle relevanten Informationen wie Titel, Beschreibung, Deadline, Wichtigkeit und Aufwand eingegeben. Nach der Verarbeitung werden die priorisierten Aufgaben übersichtlich dargestellt. Besonderer Wert wurde dabei auf eine reduzierte und klare Darstellung gelegt, um die Ergebnisse der KI leicht verständlich zu präsentieren.

Das Backend übernimmt die serverseitige Verarbeitung der Nutzereingaben. Es ist mit dem Framework FastAPI implementiert und fungiert als zentrale Steuerungseinheit des Systems. Das Backend validiert die eingehenden Daten, verwaltet die Aufgaben innerhalb des Prototyps und koordiniert die Kommunikation zwischen Frontend und KI-Modul. Darüber hinaus ist es für die Aufbereitung der Ergebnisse zuständig, die anschließend an das Frontend zurückgegeben werden.

Das KI-Modul bildet den Kern von DevFlowAI. Es analysiert die eingegebenen Aufgaben und berechnet auf dieser Grundlage einen Prioritätswert. Hierzu werden sowohl strukturierte Daten (z. B. Wichtigkeit, Aufwand, Deadline) als auch unstrukturierte Daten in Form von natürlichsprachlichen Beschreibungen berücksichtigt. Die Analyse erfolgt über eine Kombination aus einfacher Textanalyse und einem heuristischen Scoring-Modell. Zusätzlich erzeugt das KI-Modul erklärende Informationen, die nachvollziehbar machen, wie sich der jeweilige Prioritätswert zusammensetzt.

DevFlowAI ist bewusst als leichtgewichtiger Prototyp umgesetzt. Auf eine persistente Datenspeicherung oder die Anbindung externer Systeme wurde verzichtet, um den Fokus auf die Kernfunktionalität und die KI-Integration zu legen. Die Architektur ist jedoch so gestaltet, dass zukünftige Erweiterungen, etwa durch den Einsatz trainierter Modelle oder externer Schnittstellen, grundsätzlich möglich sind.

4. Systemarchitektur

Die Architektur von DevFlowAI ist als klassische Drei-Schichten-Architektur ausgelegt und besteht aus den Komponenten Frontend, Backend und KI-Modul. Ziel dieser Architektur ist eine klare Trennung der Verantwortlichkeiten sowie eine hohe Verständlichkeit des Systems. Der modulare Aufbau ermöglicht es, einzelne Komponenten unabhängig voneinander zu erweitern oder auszutauschen.

4.1 Frontend

Das Frontend stellt die Benutzerschnittstelle von DevFlowAI dar. Es ist als einfache Webanwendung mit HTML und CSS umgesetzt und verfolgt bewusst einen minimalistischen Ansatz. Dadurch liegt der Fokus auf der Funktionalität und der Darstellung der KI-Ergebnisse und nicht auf komplexem Design.

Über ein Formular können Nutzer Aufgaben erfassen und relevante Informationen eingeben. Nach der Verarbeitung durch das Backend werden die priorisierten Aufgaben zusammen mit ihrem Prioritätswert und einer Begründung angezeigt. Zusätzlich wird eine Aufschlüsselung der einzelnen Bewertungsfaktoren dargestellt, um die Entscheidungen des KI-Moduls transparent zu machen. Das Frontend kommuniziert ausschließlich über HTTP mit dem Backend und enthält keine eigene Geschäftslogik. Diese klare Trennung vereinfacht die Wartung und erhöht die Nachvollziehbarkeit der Gesamtarchitektur.

4.2 Backend

Das Backend fungiert als zentrale Verarbeitungseinheit des Systems und ist mit dem Python-Framework FastAPI implementiert. Es stellt die notwendigen Endpunkte zur Verfügung, um Aufgaben entgegenzunehmen, zu verarbeiten und die Ergebnisse an das Frontend zurückzugeben.

Zu den Hauptaufgaben des Backends gehören:

- Validierung der vom Nutzer eingegebenen Daten
- Verwaltung der Aufgaben innerhalb des Prototyps
- Weiterleitung der Daten an das KI-Modul
- Aufbereitung und Rückgabe der Priorisierungsergebnisse

Die Kommunikation zwischen Frontend und Backend erfolgt synchron über HTTP-Requests. Für den Prototyp wird eine In-Memory-Datenhaltung verwendet, wodurch auf eine persistente Datenbank verzichtet werden kann. Dies reduziert die technische Komplexität und ist für den Demonstrationszweck ausreichend. Die Validierung erfolgt durch HTML-Constraints im Frontend sowie durch Typkonvertierung im Backend.

4.3 KI-Modul

Das KI-Modul ist logisch vom Backend getrennt und kapselt die gesamte Priorisierungslogik. Es analysiert die eingehenden Aufgaben und berechnet für jede Aufgabe einen Prioritätswert. Dabei werden mehrere Faktoren berücksichtigt, darunter Dringlichkeit, Wichtigkeit, Risiko und Aufwand.

Die Analyse der Aufgabenbeschreibung erfolgt mittels einfacher NLP-Techniken, bei denen bestimmte Schlüsselbegriffe identifiziert werden. Diese Informationen fließen zusammen mit den strukturierten Metadaten in ein heuristisches Scoring-Modell ein. Das Ergebnis ist ein numerischer Score, der die Priorität der Aufgabe repräsentiert.

Ein zentrales Merkmal des KI-Moduls ist die Erklärbarkeit der Entscheidungen. Neben dem Gesamtwert wird eine Aufschlüsselung der einzelnen Faktoren erzeugt, sodass nachvollziehbar bleibt, warum eine Aufgabe höher oder niedriger priorisiert wurde.

4.4 Zusammenspiel der Komponenten

Der Ablauf innerhalb des Systems lässt sich wie folgt beschreiben:

1. Der Nutzer gibt eine Aufgabe über das Frontend ein.
2. Das Backend empfängt die Anfrage und validiert die Daten.
3. Die validierten Daten werden an das KI-Modul weitergeleitet.
4. Das KI-Modul berechnet Prioritätswerte und Erklärungen.
5. Das Backend gibt die Ergebnisse an das Frontend zurück.
6. Das Frontend stellt die priorisierte Aufgabenliste dar.

Diese klare Abfolge verdeutlicht den Datenfluss im System und unterstützt das Verständnis der Gesamtarchitektur.

Architektur:

Nutzer → Frontend → Backend → KI-Modul → Backend → Frontend (Priorisierte Aufgabenliste)

5. KI-Ansatz

Der KI-Ansatz von DevFlowAI ist darauf ausgelegt, eine sinnvolle und nachvollziehbare Entscheidungsunterstützung für die Aufgabenpriorisierung bereitzustellen. Da es sich um einen Prototyp handelt, wurde bewusst ein leichter, erklärbarer und modularer KI-Ansatz gewählt, der sich gut für Demonstrations- und Lehrzwecke eignet.

Das System folgt einem hybriden KI-Ansatz, der regelbasierte Komponenten mit KI-inspirierten

Bewertungsmodellen kombiniert. Dadurch können sowohl strukturierte als auch unstrukturierte Informationen berücksichtigt werden.

5.1 Verarbeitung strukturierter Daten

Ein Teil der Eingabedaten liegt in strukturierter Form vor. Dazu gehören insbesondere:

- die Deadline einer Aufgabe
- die vom Nutzer angegebene Wichtigkeit
- der geschätzte Aufwand

Diese Daten werden direkt numerisch verarbeitet. Die Deadline wird in eine Dringlichkeitsbewertung umgerechnet, indem die verbleibenden Tage bis zum Fälligkeitsdatum bestimmt werden. Je näher die Deadline liegt, desto höher fällt der entsprechende Dringlichkeitswert aus.

Die Attribute Wichtigkeit und Aufwand werden als gewichtete Faktoren in das Priorisierungsmodell integriert. Während eine hohe Wichtigkeit den Prioritätswert erhöht, wirkt sich ein hoher Aufwand negativ auf den Gesamtwert aus, da umfangreiche Aufgaben tendenziell mehr Zeit binden.

5.2 Analyse unstrukturierter Daten (NLP)

Neben strukturierten Metadaten verarbeitet DevFlowAI auch unstrukturierte Textdaten in Form von Aufgabenbeschreibungen. Diese Beschreibungen werden mithilfe einfacher Natural-Language-Processing-Techniken analysiert [3].

Im Rahmen des Prototyps erfolgt die Textanalyse keyword-basiert. Bestimmte Schlüsselbegriffe, die auf technische Risiken oder kritische Probleme hinweisen, werden identifiziert. Beispiele hierfür sind Begriffe wie „Bug“, „Crash“ oder „Security“. Keywords sind aktuell englisch, da typische Issue-Texte häufig englische Begriffe enthalten. Enthält eine Aufgabenbeschreibung solche Begriffe, wird der Aufgabe ein höheres Risikolevel zugewiesen. Begriffe wie „Refactor“ oder „Documentation“ werden hingegen als weniger kritisch bewertet.

Dieser Ansatz erlaubt eine erste semantische Interpretation von Texten und stellt eine einfache, aber effektive Form der NLP-Integration dar.

5.3 Scoring- und Priorisierungsmodell

Die eigentliche Priorisierung erfolgt über ein heuristisches Scoring-Modell, das verschiedene Einflussfaktoren kombiniert. Für jede Aufgabe wird ein numerischer Prioritätswert berechnet, der sich aus mehreren gewichteten Komponenten zusammensetzt.

Das verwendete Modell lässt sich vereinfacht wie folgt darstellen:

Prioritätswert =
 $0.4 \times \text{Dringlichkeit (Deadline)}$
 $+ 0.3 \times \text{Wichtigkeit}$
 $+ 0.2 \times \text{Risiko}$
 $- 0.1 \times \text{Aufwand}$

Die Gewichtung der einzelnen Faktoren wurde bewusst so gewählt, dass zeitkritische und risikobehaftete Aufgaben bevorzugt werden. Gleichzeitig wird berücksichtigt, dass sehr aufwendige Aufgaben nicht automatisch höchste Priorität erhalten.

Das Scoring-Modell ist deterministisch und nachvollziehbar, was für einen Prototyp und für erklärbare KI-Anwendungen von besonderer Bedeutung ist.

5.4 Erklärbarkeit der KI-Entscheidungen

Ein zentrales Ziel von DevFlowAI ist die Transparenz der KI-Entscheidungen. Daher wird nicht nur der finale Prioritätswert ausgegeben, sondern zusätzlich eine Aufschlüsselung der einzelnen Bewertungsfaktoren bereitgestellt.

Für jede Aufgabe kann nachvollzogen werden:

- welchen Beitrag die Deadline zur Priorität leistet
- wie stark Wichtigkeit und Risiko gewichtet wurden
- in welchem Umfang der Aufwand den Wert reduziert

Dieser Ansatz entspricht dem Konzept der Explainable AI (XAI) und erhöht das Vertrauen der Nutzer in die vom System erzeugten Ergebnisse.

5.5 Einordnung des KI-Ansatzes

Der in DevFlowAI eingesetzte Ansatz stellt kein trainiertes maschinelles Lernmodell dar. Vielmehr handelt es sich um ein KI-inspiriertes Entscheidungsunterstützungssystem, das klassische KI-Methoden wie heuristische Bewertung und einfache NLP-Techniken nutzt.

Diese bewusste Einschränkung ist für einen Prototyp angemessen und ermöglicht es, die grundlegenden Prinzipien von KI-Systemen - Analyse, Bewertung, Entscheidungsfindung und Erklärbarkeit - klar zu demonstrieren. Gleichzeitig ist die Architektur so gestaltet, dass der KI-Ansatz in zukünftigen Iterationen durch trainierte Modelle oder vortrainierte Sprachmodelle ersetzt oder erweitert werden kann.

6. Implementierung

Die Implementierung von DevFlowAI erfolgte mit dem Ziel, einen lauffähigen und verständlichen

Prototypen zu realisieren, der die konzipierte Architektur und den KI-Ansatz praktisch umsetzt. Dabei wurde bewusst auf eine geringe technische Komplexität geachtet, um den Fokus auf die Kernfunktionalitäten und die KI-Integration zu legen.

6.1 Technologiewahl

Für die Umsetzung des Prototyps wurde die Programmiersprache Python gewählt. Python eignet sich besonders gut für die Entwicklung von KI-gestützten Anwendungen, da es eine hohe Lesbarkeit bietet und zahlreiche Bibliotheken für Webentwicklung und KI-Anwendungen zur Verfügung stellt.

Als Webframework kommt FastAPI zum Einsatz. FastAPI ermöglicht eine schnelle Entwicklung von Webanwendungen, bietet eine klare Struktur für REST-Endpunkte und unterstützt moderne Python-Standards [5]. Für die Darstellung der Benutzeroberfläche wird Jinja2 zur serverseitigen Template-Generierung verwendet [7]. Die eigentliche Oberfläche besteht aus einfachem HTML und CSS.

6.2 Backend-Implementierung

Das Backend bildet das zentrale Steuerungselement des Systems. Es verarbeitet alle Nutzereingaben und koordiniert die Interaktion zwischen Frontend und KI-Modul. Die wichtigsten Funktionen des Backends umfassen:

- Entgegennahme und Validierung der Formulardaten
- Verwaltung der Aufgaben innerhalb des Prototyps
- Aufruf des KI-Moduls zur Prioritätsberechnung
- Rückgabe der Ergebnisse an das Frontend

Für den Prototyp werden die Aufgaben in einer In-Memory-Datenstruktur gespeichert. Dies erlaubt eine einfache und schnelle Verarbeitung, ohne die zusätzliche Komplexität einer Datenbankanbindung einzuführen.

6.3 KI-Modul

Das KI-Modul ist als eigenständige Komponente implementiert und kapselt die gesamte Priorisierungslogik.

Es besteht aus mehreren Teilschritten:

1. Berechnung der verbleibenden Zeit bis zur Deadline
2. Analyse der Aufgabenbeschreibung mittels keyword-basierter NLP-Techniken
3. Berechnung der Einzelbewertungen für Dringlichkeit, Risiko, Wichtigkeit und Aufwand
4. Zusammenführung der Einzelwerte in einem gewichteten Scoring-Modell

Die Implementierung ist modular aufgebaut, sodass einzelne Bestandteile - etwa die Textanalyse

oder das Scoring-Modell - leicht angepasst oder ausgetauscht werden können.

6.4 Frontend-Implementierung

Das Frontend ist bewusst minimalistisch gestaltet und konzentriert sich auf die Kernfunktionalitäten. Über ein Formular können Nutzer neue Aufgaben anlegen und deren Attribute erfassen. Nach der Verarbeitung durch das Backend werden die priorisierten Aufgaben in Form einer übersichtlichen Liste dargestellt.

Zusätzlich werden erklärende Informationen ausgegeben, die zeigen, wie sich der Prioritätswert einer Aufgabe zusammensetzt. Dies unterstützt die Nachvollziehbarkeit der KI-Entscheidungen und entspricht dem Anspruch einer erklärbaren KI.

6.5 Integration und Ablauf

Der typische Ablauf innerhalb der implementierten Anwendung gestaltet sich wie folgt:

1. Der Nutzer gibt eine Aufgabe über das Webformular ein.
2. Das Backend verarbeitet die Eingabe und ruft das KI-Modul auf.
3. Das KI-Modul berechnet Prioritätswerte und Begründungen.
4. Die Ergebnisse werden an das Frontend zurückgegeben.
5. Das Frontend stellt die priorisierte Aufgabenliste dar.

Dieser Ablauf verdeutlicht, wie die einzelnen Komponenten im Rahmen der Implementierung zusammenarbeiten.

6.6 Besonderheiten der Implementierung

Ein besonderer Fokus der Implementierung liegt auf der Transparenz der Entscheidungen. Neben dem finalen Prioritätswert werden zusätzliche Informationen zur Berechnung angezeigt. Dadurch können Nutzer die Entscheidungen des Systems nachvollziehen und kritisch hinterfragen. Darüber hinaus wurde die Implementierung bewusst so gestaltet, dass sie leicht erweiterbar ist. Zukünftige Erweiterungen, wie der Einsatz trainierter Modelle oder die Integration externer Systeme, können ohne grundlegende Änderungen an der Architektur realisiert werden.

7. Evaluation

Die Evaluation von DevFlowAI erfolgt qualitativ und orientiert sich an den Zielsetzungen des Projekts. Da es sich um einen Prototyp handelt, liegt der Fokus nicht auf einer quantitativen Messung der Modellgüte, sondern auf der Bewertung der Funktionalität, Nachvollziehbarkeit und Eignung des Systems als KI-gestützte Entscheidungsunterstützung.

7.1 Funktionale Evaluation

Der entwickelte Prototyp erfüllt die vorgesehenen funktionalen Anforderungen. Nutzer können

Aufgaben über die Weboberfläche erfassen und erhalten unmittelbar eine priorisierte Aufgabenliste. Die Anwendung reagiert stabil auf unterschiedliche Eingabewerte und verarbeitet sowohl strukturierte Daten (z. B. Wichtigkeit, Aufwand, Deadline) als auch unstrukturierte Textdaten in Form von Aufgabenbeschreibungen.

Besonders hervorzuheben ist, dass Aufgaben mit naher Deadline oder hohem technischem Risiko zuverlässig höher priorisiert werden als weniger kritische Aufgaben. Damit entspricht das Verhalten des Systems den Erwartungen aus dem definierten Use Case.

7.2 Bewertung des KI-Ansatzes

Der eingesetzte KI-Ansatz erweist sich für einen Prototyp als zweckmäßig. Durch die Kombination aus einfacher Textanalyse und heuristischem Scoring-Modell gelingt es, relevante Einflussfaktoren systematisch zu berücksichtigen und in eine konsistente Priorisierungsentscheidung zu überführen.

Obwohl kein trainiertes maschinelles Lernmodell eingesetzt wird, zeigt das System typische Eigenschaften KI-gestützter Entscheidungsunterstützung:

- Verarbeitung unstrukturierter Eingaben
- Gewichtete Bewertung mehrerer Einflussfaktoren
- Ableitung einer Empfehlung statt einer harten Entscheidung

Damit erfüllt DevFlowAI die Anforderungen an ein KI-System auf Prototyp-Ebene.

7.3 Erklärbarkeit und Transparenz

Ein zentraler Aspekt der Evaluation ist die Erklärbarkeit der erzeugten Ergebnisse. DevFlowAI stellt neben dem finalen Prioritätswert auch eine Begründung sowie eine Aufschlüsselung der einzelnen Bewertungsfaktoren bereit.

Diese Transparenz ermöglicht es Nutzern, die Entscheidungen des Systems nachzuvollziehen und kritisch zu hinterfragen. Gerade im Kontext von KI-Systemen ist diese Eigenschaft von großer Bedeutung, da sie das Vertrauen in die Ergebnisse erhöht und eine informierte Nutzung unterstützt [2].

7.4 Nutzerperspektive

Aus Sicht der Nutzer bietet DevFlowAI einen klaren Mehrwert als Entscheidungsunterstützung. Die Weboberfläche ist einfach gehalten und ermöglicht eine schnelle Eingabe von Aufgaben. Die Ergebnisse sind übersichtlich dargestellt und lassen sich leicht interpretieren.

Der Prototyp eignet sich insbesondere für Einzelentwickler oder kleine Teams, die eine

strukturierte Unterstützung bei der Aufgabenpriorisierung suchen. Durch die Möglichkeit, Demo-Daten zu laden, lässt sich das System zudem gut für Präsentations- und Demonstrationszwecke einsetzen.

7.5 Gesamteinschätzung

Insgesamt zeigt die Evaluation, dass DevFlowAI die gesteckten Projektziele erreicht. Der Prototyp demonstriert erfolgreich, wie KI-basierte Methoden zur Unterstützung von Priorisierungsentscheidungen in der Softwareentwicklung eingesetzt werden können.

Die Kombination aus funktionaler Umsetzung, nachvollziehbarer KI-Logik und klarer Architektur macht das System zu einem geeigneten Beispiel für den Einsatz von KI in einem praxisnahen Anwendungsszenario [4].

8. Limitationen

Trotz der erfolgreichen Umsetzung des Prototyps weist DevFlowAI mehrere Einschränkungen auf, die bei der Bewertung der Ergebnisse berücksichtigt werden müssen. Diese Limitationen sind größtenteils bewusst gewählt und ergeben sich aus dem prototypischen Charakter des Systems sowie aus dem Fokus auf Verständlichkeit und Demonstrationsfähigkeit.

Eine zentrale Einschränkung besteht darin, dass kein trainiertes maschinelles Lernmodell eingesetzt wird. Die Priorisierung basiert auf einem heuristischen Scoring-Modell mit fest definierten Gewichtungen. Dadurch kann das System nicht aus historischen Daten lernen oder seine Entscheidungen automatisch an veränderte Rahmenbedingungen anpassen.

Die Analyse der Aufgabenbeschreibungen erfolgt ausschließlich keyword-basiert. Zwar erlaubt dieser Ansatz eine einfache semantische Interpretation der Texte, er ist jedoch begrenzt in seiner Ausdruckskraft. Synonyme, komplexe Satzstrukturen oder implizite Bedeutungen können nicht zuverlässig erkannt werden. Dadurch besteht die Gefahr, dass das technische Risiko einzelner Aufgaben falsch eingeschätzt wird.

Darüber hinaus verfügt DevFlowAI über keine persistente Datenspeicherung. Alle Aufgaben werden ausschließlich im Arbeitsspeicher gehalten und gehen beim Neustart der Anwendung verloren. Für den Prototyp ist dieses Vorgehen ausreichend, für einen produktiven Einsatz wäre jedoch eine Datenbankbindung erforderlich.

Ein weiterer limitierender Faktor ist das Fehlen externer Systemintegrationen. Der Prototyp verarbeitet ausschließlich manuell eingegebene Aufgaben. Eine automatische Übernahme von Daten aus Entwicklerwerkzeugen wie Issue-Trackern oder Versionsverwaltungssystemen ist nicht

implementiert.

Schließlich ist die Evaluation des Systems rein qualitativ. Es wurden keine quantitativen Metriken oder Vergleichsstudien durchgeführt, um die Qualität der Priorisierungsentscheidungen objektiv zu messen. Für ein Forschungs- oder Produktivsystem wäre eine solche Evaluation jedoch notwendig.

Zusammenfassend sind die genannten Limitationen für einen akademischen Prototyp akzeptabel und teilweise bewusst gewählt. Sie verdeutlichen gleichzeitig die Grenzen des aktuellen Systems und liefern Ansatzpunkte für zukünftige Erweiterungen.

9. Ausblick

Der entwickelte Prototyp DevFlowAI bietet eine solide Grundlage für weiterführende Erweiterungen und Verbesserungen. Aufbauend auf der bestehenden Architektur und dem aktuellen KI-Ansatz ergeben sich mehrere sinnvolle Weiterentwicklungsmöglichkeiten, die das System funktional und methodisch vertiefen könnten.

Ein naheliegender nächster Schritt ist der Einsatz fortgeschrittener NLP-Verfahren. Anstelle der keyword-basierten Textanalyse könnten vortrainierte Sprachmodelle oder sogenannte Sentence Embeddings verwendet werden, um Aufgabenbeschreibungen semantisch präziser zu erfassen [6]. Dadurch ließen sich Bedeutungszusammenhänge, Synonyme und kontextuelle Informationen besser berücksichtigen, was zu einer genaueren Risikobewertung führen würde.

Darüber hinaus könnte das heuristische Scoring-Modell durch ein lernendes maschinelles Lernverfahren ersetzt oder ergänzt werden. Auf Basis historischer Priorisierungsentscheidungen oder Nutzerfeedbacks wäre es möglich, die Gewichtungen der einzelnen Faktoren automatisch anzupassen. Ein solches Modell könnte sich im Laufe der Zeit an individuelle Arbeitsweisen oder projektbezogene Anforderungen anpassen.

Ein weiterer Erweiterungspunkt betrifft die Integration externer Systeme. Die Anbindung an Issue-Tracking-Systeme oder Versionsverwaltungen würde es ermöglichen, Aufgaben automatisch zu importieren und in Echtzeit zu priorisieren [9]. Dies würde den manuellen Eingabeaufwand reduzieren und den Praxisnutzen des Systems deutlich erhöhen. Eine mögliche Erweiterung ist die Anbindung an Issue-Tracker wie GitLab über dessen REST-API [8].

Auch im Bereich der Benutzerinteraktion bestehen Erweiterungspotenziale. So könnten personalisierte Profile eingeführt werden, die unterschiedliche Priorisierungsstrategien für verschiedene Nutzer oder Projekte erlauben. Zusätzlich wäre eine persistente Datenspeicherung

notwendig, um Aufgaben und Priorisierungsverläufe langfristig zu speichern und auszuwerten.

Schließlich könnte die Evaluation des Systems durch quantitative Methoden ergänzt werden. Beispielsweise könnten Priorisierungsergebnisse mit menschlichen Entscheidungen verglichen oder Nutzerstudien durchgeführt werden, um den praktischen Nutzen und die Akzeptanz des Systems systematisch zu untersuchen.

Insgesamt zeigt der Ausblick, dass DevFlowAI über den aktuellen Prototyp hinaus ein hohes Erweiterungspotenzial besitzt und als Ausgangspunkt für weiterführende Forschungs- oder Entwicklungsarbeiten dienen kann.

10. Fazit

Im Rahmen dieses Projekts wurde mit DevFlowAI ein prototypisches KI-gestütztes System zur Aufgabenpriorisierung für Softwareentwickler konzipiert und implementiert. Ziel war es, zu untersuchen, wie Methoden der Künstlichen Intelligenz sinnvoll in eine praxisnahe Anwendung integriert werden können, um Entwickler bei Entscheidungsprozessen zu unterstützen.

Der entwickelte Prototyp zeigt, dass bereits einfache KI-Ansätze, wie die Kombination aus heuristischer Bewertung und grundlegender Textanalyse, einen Mehrwert für die Priorisierung von Entwicklungsaufgaben bieten können. Besonders hervorzuheben ist die Transparenz der Entscheidungsfindung, da das System seine Priorisierungen nachvollziehbar begründet und die einzelnen Einflussfaktoren offenlegt.

Die modulare Systemarchitektur ermöglicht eine klare Trennung von Frontend, Backend und KI-Modul und erleichtert sowohl das Verständnis als auch mögliche Erweiterungen. Trotz der bewusst gewählten Vereinfachungen erfüllt DevFlowAI alle Anforderungen eines KI-Prototyps im akademischen Kontext und demonstriert anschaulich die zentralen Konzepte KI-gestützter Entscheidungsunterstützung.

Zusammenfassend lässt sich festhalten, dass DevFlowAI die Projektziele erfolgreich erreicht und eine geeignete Grundlage für weiterführende Arbeiten im Bereich KI-gestützter Softwarewerkzeuge darstellt.

11. Quellen

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[2] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," arXiv preprint arXiv:1702.08608, 2017. [Online]. Verfügbar hier: <https://arxiv.org/abs/1702.08608>.

[Zugegriffen am: 11.01.2026].

[3] D. Jurafsky and J. H. Martin, Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models, 3rd ed., online manuscript, 2026. [Online]. Verfügbar hier: <https://web.stanford.edu/~jurafsky/slp3/>. [Zugegriffen am: 12.01.2026].

[4] D. J. Power, Decision Support Systems: Concepts and Resources for Managers. Westport, CT, USA: Quorum Books, 2002.

[5] FastAPI Documentation. [Online]. Verfügbar hier: <https://fastapi.tiangolo.com/>. [Zugegriffen am: 12.01.2026].

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," arXiv preprint arXiv:1908.10084, 2019. [Online]. Verfügbar hier: <https://arxiv.org/abs/1908.10084>. [Zugegriffen am: 12.01.2026].

[7] Jinja Documentation (The Pallets Projects). [Online]. Verfügbar hier: <https://jinja.palletsprojects.com/en/stable/>. [Zugegriffen am: 12.01.2026].

[8] GitLab REST API Documentation. [Online]. Verfügbar hier: <https://docs.gitlab.com/api/rest/>. [Zugegriffen am: 12.01.2026].

[9] GitLab Issues API Documentation. [Online]. Verfügbar hier: <https://docs.gitlab.com/api/issues/>. [Zugegriffen am: 12.01.2026].

12. Ergänzende Informationen

12.1 Gitlab Link von DevFlowAI:

<https://git.mylab.th-luebeck.de/blend.salihu/DevFlowAI.git>